AI Applications in Empirical Research

Concepts, Workflows, and the Use of Claude Code

A concept summary

September 2026

Abstract

This document summarizes the main concepts surrounding the use of AI agents—and Claude Code in particular—in quantitative empirical research. It draws on the ongoing writing of economist Scott Cunningham (*Scott's Mixtape Substack*), who has documented in real time how coding agents change the production, verification, and understanding of research. The aim is a compact conceptual map: what these tools are, what they can and cannot do autonomously, the economic logic of why they matter, and the workflow discipline (checklists, hooks, skills, connectors) needed to use them responsibly. Attributions of specific framings and coinages are to Cunningham unless noted; errors in summarization are the summarizer's own.

Contents

1 From chatbots to coding agents **1**

2 The core capability: "3-D printing" a research paper **1**

3 The organizing framework: production, verification, understanding **2**
3.1 Verification is subtle 2
3.2 Understanding is the real bottleneck 2

4 The economic logic: why agents change research output **3**

5 Workflow discipline: the checklist and the "harness" **3**
5.1 A concrete example: the difference-in-differences harness 3

6 Claude Code mechanics for research **4**

7 Pedagogy: teaching agents for research **4**

8 Open problems and the human in the loop **5**

9 Summary **5**

1 From chatbots to coding agents

A useful first distinction is that *AI agents for research are not chatbots.* Language-model tools fall into three broad "harnesses":

- **Chatbots** — e.g. ChatGPT and Claude, used conversationally through a graphical interface.
- **Coding copilots** — e.g. Cursor and GitHub Copilot, which autocomplete and assist inside an editor.
- **Coding agents** — e.g. Claude Code and Codex, which act with agency across the file system and tools of the computer: reading and writing files, running shell commands, downloading data, and executing multi-step plans.

The practical significance of the third category is that it accesses the *entire hard drive and toolchain* of the machine. For social scientists—who historically worked through the GUIs of Stata or RStudio rather than a terminal—the arrival of a coding agent inside a desktop app (rather than only a command-line interface) is what turned adoption from a software-engineering phenomenon into an empirical-research one.

2 The core capability: "3-D printing" a research paper

Contemporary coding agents can, start to finish and largely autonomously, produce a submittable quantitative manuscript: articulate a research question, choose an identification strategy, locate and download datasets, select estimators, write replicable code, generate tables and figures with robustness checks, and draft a full paper with references and appendices—often in well under an hour. Cunningham calls this "3-D printing a research paper." The existence of large-scale autonomous pipelines (e.g. the Social Catalyst Lab's automated program-evaluation project reporting on the order of a thousand generated empirical papers) shows this is no longer a conjecture: *agents can autonomously produce empirical research.*

But not full self-driving. The right analogy is not a fully autonomous vehicle but *lane-keeping plus cruise control.* The agent can produce, yet it should not be left to drive unsupervised, because production alone does not guarantee a correct or well-understood result.

3 The organizing framework: production, verification, understanding

The central conceptual contribution is to treat research with agents as having to pass through **three distinct "doors"**:

1. **Production** — generating the analysis, code, exhibits, and manuscript. This is the door agents already handle well.
2. **Verification** — confirming the output is accurate: that what was claimed to be done was in fact done, *and* that what should not have been done was not done.
3. **Understanding** — the researcher's own warranted comprehension of what was done, why, and what it means.

Before agents, these three were bundled together almost by accident: the act of producing research produced verification and understanding as by-products. Agents *unbundle* them. The researcher is now less an individual craftsperson and more the **CEO of a research enterprise**—responsible for knowing, in an approximately accurate sense, that tasks were done, by whom, and with what result, even without performing each task personally.

3.1 Verification is subtle

Verification is not merely re-running code in several languages or checking that every figure was produced by stored code. Those are *necessary but not sufficient*. Verification is endogenous to a *pre-specified target estimand*: it checks that the analysis pursued the estimand set out ex ante and nothing else. Agents can confirm what *was* done, but they cannot confirm that what should *not* have been done was avoided—"there is nothing to verify." A documented failure mode is *endogenous sorting into specifications* in response to true statements in a project's context, not just in response to explicit instructions.

3.2 Understanding is the real bottleneck

A popular slogan holds that "verification is the new bottleneck." The sharper claim is that **understanding is the new bottleneck**: agents may eventually verify as well as they produce, but that does not make the human understand the result. If understanding was historically an *externality* of production—created almost as a by-product of doing the work—then removing the human from production may quietly remove understanding too, leaving "cognitive debt" and a principal–agent problem in which the human trusts an agent whose work they cannot independently judge.

4 The economic logic: why agents change research output

The shift is naturally expressed with elementary price theory. Let the demand for an individual's research output be downward sloping. Before AI, research was produced up to the point where marginal benefit equals a (relatively high) marginal cost of production, MC_b, fixed by time and skill:

$$MB(Q) = MC_b \implies Q_b^*.$$

Agents drive the marginal cost of production down to some small $\widehat{MC}_{AI} = \varepsilon \approx 0$, so

$$MB(Q) = \widehat{MC}_{AI} \implies Q_{AI}^* \gg Q_b^*.$$

Output rises, and additional surplus is generated both on the old work (done with fewer time inputs) and on the newly feasible work. The open question is what happens to the *quality* component of the old surplus: whether the time freed up can actually be re-invested to restore the understanding that production used to supply for free. This is why the analysis returns to verification and understanding as binding constraints rather than free goods.

Zero errors as a constraint, not an objective. A key discipline: treat "zero errors" as a *constraint* in the researcher's optimization, not as an objective to be minimized. Minimizing errors implicitly licenses "optimal errors"; a constraint does not. Formally, one produces the manuscript subject to passing production, verification, and understanding with a zero-error requirement.

5 Workflow discipline: the checklist and the "harness"

Because experts skip steps (Kahneman's fast "System 1" thinking) precisely on high-stakes tasks, the proposed antidote is a **checklist**, in the spirit of Atul Gawande's *The Checklist Manifesto*. A *harness* is a checklist-centered piece of software that stages empirical work so that every project is pushed deliberately through production, verification, and understanding. Design principles—credited to the tradition of Rubin, Imbens, and Sant'Anna ("design trumps analysis")—anchor the checklist.

5.1 A concrete example: the difference-in-differences harness

Rather than a universal tool, the harness is deliberately *narrow* (difference-in-differences), because a narrow scope is easier to evaluate and perfect. Two illustrative checklist stages:

- **Packages** — a visual card recording exactly which packages and versions the project uses.
- **"Bite"** — an exhaustive descriptive study of the very first thing the intervention does in the world (e.g. for a gun law, the movement of guns; for a minimum wage, where it binds), across the two dimensions of *space* (heat maps) and *time* (series aggregations), for treated and comparison groups alike.

Crucially, **"bite" is the stage *not* to fully automate**. Its purpose is to build the researcher's own insight into the *treatment assignment mechanism*, which is what design-based inference (following Angrist and Imbens) demands. The main assignment mechanisms to reason about:

- **Human rationality / essential heterogeneity** (endogenous, self-selected treatment) — the hard case that makes naive causal inference unreliable (cf. LaLonde 1986).
- **Instruments** — assign treatment to compliers; motivate instrumental variables and the LATE framework (Imbens & Angrist 1994), with randomization "buried" in the instrument.
- **Randomization** — direct experimental assignment on the treatment itself (RCTs).
- **Running variables** — threshold-based assignment motivating regression discontinuity (cf. Imbens & Lemieux's guide to practice).

Agents happily automate the *mechanics* of bite (maps, code, time series, data wrangling); the researcher retains the *detective work* of interpreting the assignment mechanism and choosing a credible design and comparison group.

6 Claude Code mechanics for research

Markdown / project memory
Plain-text instruction and context files (e.g. a `CLAUDE.md`) that persist project conventions and guidance the agent reads on each run.

Hooks
Deterministic, event-triggered actions the harness executes around tool calls—used here to enforce accuracy-preserving steps such as "don't edit raw data," or to wire figure/table production into a deck or manuscript. The recommendation is *specific* hooks tied to each checklist step rather than abstract, universal ones.

Skills
Packaged, reusable procedures the agent can invoke. Example: a custom `/quiz` skill that has the agent generate teaching slides and then ask the researcher multiple-choice questions about the work—modeled on corporate "training-video quizzes"—to manufacture *continuous understanding* on top of verification.

Connectors and MCP
Connectors extend the agent beyond the local machine into cloud services; the *Model Context Protocol (MCP)* is the open standard—"new piping"—that lets AI applications securely connect to external data sources and tools. Connectors work from the phone app as well as Claude Code, and

range from official to community-built ("buyer beware"). For research they are still exploratory; their clearer near-term value is operational (e.g. a solo-entrepreneur running bookkeeping/payroll via QuickBooks, Gusto, and Stripe connectors).

The `git diff` as unit of verification
Reviewing each diff is a natural verification primitive, but skimming and accepting diffs can silently accrue cognitive debt—hence the push to pair diffs with active understanding checks (the quiz).

7 Pedagogy: teaching agents for research

A recommended talk structure for empirical audiences: (1) history of thought, (2) narrative, (3) numerical examples, (4) an interactive demo, (5) then technical rigor. The signature device is the **"magic trick"**: open on an *empty folder* ("nothing up my sleeve"), prompt the agent to conduct a full autonomous study, and let it run for 20–30 minutes while lecturing; the finished manuscript, figures, and deck appearing in that folder deliver the visceral lesson that agents are not chatbots—and that production can occur with *zero* human time inputs, throwing verification and understanding into relief.

A related polemic: demands that authors "submit their prompts" with a paper misunderstand the medium—using an agent as a *thinking partner* is more like collaborating with a coauthor than issuing prompts, and there is no meaningful "prompt engineering" to disclose when one simply converses naturally with the model.

8 Open problems and the human in the loop

- **Where should the human be in the loop?** Which research stages to automate, and which (like "bite") to keep manual, remains an active design question.
- **Skill atrophy & accountability.** If human skills atrophy, the researcher may lose the ability to judge whether work was done well—echoing the maxim that a machine cannot be held accountable and therefore cannot be trusted to decide.
- **Model heterogeneity.** Whether frontier models differ systematically and predictably in their fitness for autonomous research is not yet settled.
- **Interfaces for understanding.** Much of the proposed harness is deliberately *visual* (interactive, "flippable" figure cards; dashboards; even head-mounted displays), reflecting both general cognition and individual needs, to keep the researcher seeing and remembering the work.

9 Summary

AI coding agents make research *production* nearly free, which is precisely why *verification* and, above all, *understanding* become the binding constraints. The proposed response is disciplined workflow: a narrow, checklist-based harness that stages each project through the three doors; specific hooks and skills (including active understanding checks) built on top of Claude Code; and a firm "zero errors" constraint. Used this way, the agent becomes a collaborator that raises the ceiling on research quality and quantity—rather than an autopilot that produces work no one truly understands.

Primary source. Scott Cunningham, *Scott's Mixtape Substack* (https://causalinf.substack.com), 2026 essays, including "Which stages of research production should we let AI Agents loose," "How to improve understanding with Claude Code," "My strategy for talks on agents for research," and "Connectors and My Harness Idea." Concepts summarized and reorganized here; any misrepresentation is the summarizer's own.